SC3000 / CZ3005 Lab Assignment 1

Part 1 Finding a Shortest Path with An Energy Budget

Problem description

We are given a problem of finding the shortest traverse path between two locations within a certain energy budget. This scenario often happens in transportation where a shorter path is not necessarily more energy efficient (e.g., the road may be very bumpy or has heavy traffic). We illustrate our problem with the following example:

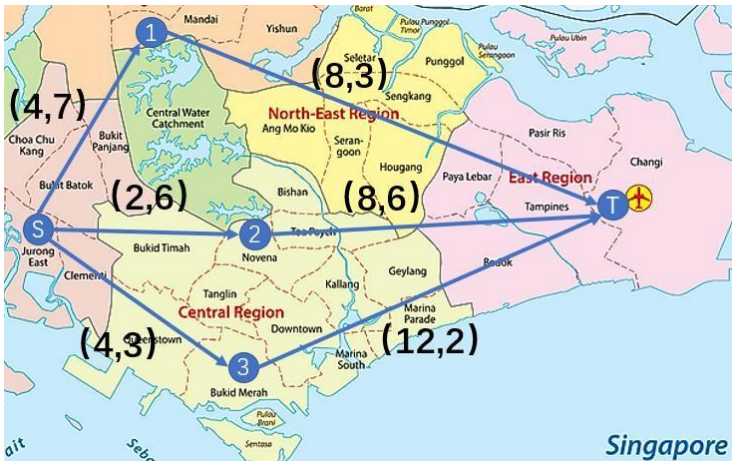


Figure 1: A small example.

Suppose we are given a graph with a predefined source node (S) and a target node (T). Each edge in the graph is associated with a pair of values (X, Y) where X denotes the distance and Y denotes the energy cost. Also see Figure 1 for an illustration. The job is to find the shortest path between S and T such that the accumulated energy cost along the path does not exceed an energy budget. Specifically, we want to find the shortest path from Jurong East to Changi Airport not exceeding an energy budget 11. We can observe that although the path S->2->T has the shortest travel distance 10, its accumulated energy cost 12 exceeds the energy budget 11. Thus, this path is infeasible. In this example, the path S->1->T is the shortest feasible path.

1.1 Problem instance

You will be given a problem instance of New York city (NYC) road network of the United State. This instance is taken and modified from the 9th DIMACS implementation challenge. This instance has 264346 nodes and 730100 edges. You will be given four python dictionary files:

1.1.1 Graph dictionary **G**
The graph is given as an adjacency list where the neighbor list of node 'v' can be accessed with **G**['v'].

1.1.2 Node coordination dictionary **Coord**
The coordination of a node 'v' is a pair (X, Y) which can be accessed with **Coord**['v']

1.1.3 Edge distance dictionary **Dist**
The distance between a pair of nodes (v, w) can be accessed with **Dist**['v,w'].

1.1.4 Edge cost dictionary **Cost**

The energy cost between a pair of nodes (v, w) can be accessed with **Cost**[‘v,w’.]

1.1 Tasks and marking criteria

You will need to solve three tasks that are listed as follows:

Task 1: You will need to solve a relaxed version of the NYC instance where we do not have the energy constraint. You can use any algorithm we discussed in the lectures. Note that this is equivalent to solving the shortest path problem. The solution quality of your algorithm will affect the grade. (15 marks)

Task 2: You will need to implement an uninformed search algorithm (e.g., the DFS, BFS, UCS) to solve the NYC instance. (15 marks)

Task 3: You will need to develop an A* search algorithm to solve the NYC instance. The key is to develop a suitable heuristic function for the A* search algorithm in this setting. The solution quality of your algorithm will affect the grade.
(10 marks)

For tasks 2 and 3, the energy budget is set to be 287932. For all the 3 tasks, the starting and ending nodes are set to be ‘1’ and ‘50’, respectively.

(Please note that the NYC instance is provided separately as formatted in section 1.1)

1.2 Output format

The output of your algorithm should be formatted as the following (using Figure 1 as an example):

Shortest path: S->1->T.
Shortest distance: 12.
Total energy cost: 10.

1.3 Programming language

For the sake of running your codes, you are required to use Python or Java. Please name your program file “main.py” or “main.java” and put the file under the main folder named with your team’s name. You should also put the instance files into your main folder. You must make sure your codes are runnable. The TAs may run your codes by simply executing the command “python main.py” or “java main”. All the algorithms must be implemented by you.

Part 2 Solving MDP and Reinforcement Learning Problems Using a Grid World

Problem description

In this assignment, you will implement and compare planning and learning approaches for sequential decision-making using a fully specified grid-based environment. You will first solve the problem as a Markov Decision Process (MDP) with a known model and then solve it as a reinforcement learning (RL) problem when the model is unknown.

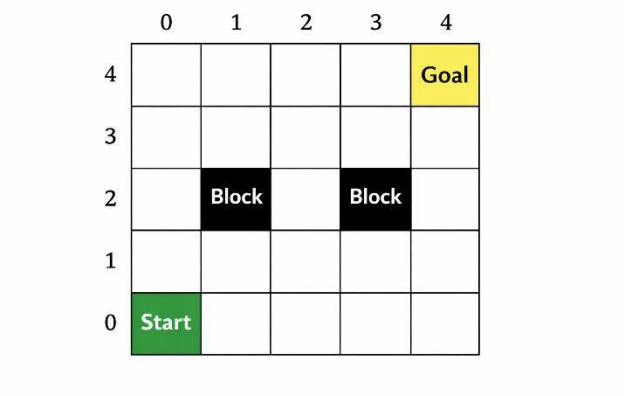
Grid World Specification

The environment is a **5 × 5 grid world**. Each grid cell corresponds to a state represented by its coordinates (x, y), where $x, y \in \{0,1,2,3,4\}$.

- **Start state:** $(0, 0)$ (bottom-left corner)
- **Goal state:** $(4, 4)$ (top-right corner)
- **Terminal state:** The episode ends when the agent reaches the goal state.
- **Roadblocks:** The agent **cannot enter cells** $(2, 1)$ and $(2, 3)$. These cells act as obstacles and must be avoided when planning or learning a policy.

At each time step, the agent can take one of four actions: move **Up**, **Down**, **Left**, or **Right**. If an action would move the agent outside the grid or into a roadblock, the agent remains in its current state.

Figure 1 shows the grid with the start, goal, and roadblocks:



Rewards

- Each action incurs a step cost of −1.
- Reaching the goal state yields a reward of +10, after which the episode ends.

Stochastic Transition Model (Example)

The environment dynamics are **stochastic**, meaning that the intended action may not always be executed as planned.

When the agent selects an action $a$in state s:

- With probability **0.8**, the agent moves in the intended direction.
- With probability **0.1**, the agent moves in a direction **perpendicular to the intended action** (to the left or right of the intended direction).
- If a resulting movement would cause the agent to leave the grid or enter a roadblock, the agent remains in the same state.

For example:

- If the agent chooses **Up**:
  - With probability 0.8, it moves **Up** ○
    With probability 0.1, it moves **Left**
  - With probability 0.1, it moves **Right**

Similar transition probabilities apply to the other actions.

2.1 Tasks and marking criteria

Task 1: The environment model is fully known. The transition dynamics are deterministic, and the discount factor is $\boldsymbol{\gamma = 0.9}$. You are required to implement two planning algorithms: (1) Value Iteration: Compute the optimal value function using the Bellman optimality equation and derive the optimal policy. (2) Policy Iteration: Start from an initial policy and alternate between policy evaluation and policy improvement until convergence. Report the resulting value functions and policies, and compare the policies obtained from value iteration and policy iteration. (20 marks)
Task 2: If the agent does not have access to the transition model, you are required to implement Monte Carlo (MC) prediction and control using complete episodes. Specifically, you are required to use an ε greedy strategy for policy improvement. Use a fixed ε value of 0.1 throughout training. Estimate state–action values using sampled returns. Train the agent over multiple episodes starting from the start state. After training, extract the learned policy and compare it with the optimal policies from Task 1. (15 marks)

Task 3: You will implement tabular Q-learning with an ε-greedy exploration strategy, as covered in the lecture. Use a fixed ε value of 0.1 throughout training. The agent interacts with the environment without knowledge of the transition model in advance. Use a fixed learning rate of $\alpha = 0.1$for updating the Q values. After implementation, you are required to compare the Q-learning policy with both the Monte Carlo–learned policy (Task 2) and the optimal policy obtained in Task 1. Provide your analysis of the convergence speed and efficiency of learning in the report. (15 marks)

2.2 Constraints and Notes

All value functions and action-value functions must be tabularly represented. Deep neural networks or function approximation methods are not allowed. Implementations should be clear, modular, and well documented.

Requirements and guidelines

1. Teaming Formation

You are required to form a team of up to 3 people to complete this project. You are required to form a team of up to **three** people to complete this project. All team members must be from the **same tutorial group**. This requirement is to facilitate **mark moderation across tutorial groups**. A team with only a single person is also allowed. For team size that is larger than one, you need to clearly state the contribution of each team member.

2. Honor Code

If you use any existing code, libraries, or AI models (such as Google Gemini or ChatGPT) etc. and consult any papers, books, online references, etc. for your project**, you must cite your sources in your report and clearly indicate which parts of the project are your contributions and which parts were implemented by others.** Using others' code/ideas without acknowledgement will lead to failure of your project.

3. Deliverables

You are required to produce:

(1) A final report of no more than 5 pages. Name your report as *<team name>-final-report.pdf*.

(2) Source codes for your project. Your source codes are to be submitted in .zip format. Name the zip file as *<team name>-final-code.zip*.

The following is a suggested structure for the report:

(1) Title, team members.
(2) For each task, explain your approach, output, etc.
(3) Contribution: Please clearly state the contribution of each team member if you have more than one member in your team.
(4) Conclusion: What have you learned?
(5) References: Please cite the sources you consulted in the report.